Local 2D SLAM with Dynamic Path Planning

Nikolai Philipenko
University of Alberta
Faculty of Engineering
Department of Electrical and Computer Engineering
nphilipe@ualberta.ca

Yuyang Wang
University of Alberta
Faculty of Science
Department of Computing Science
uyangwang@ualberta.ca

***Abstract*—Simultaneous Localization and Mapping (SLAM) is a fundamental prerequisite for autonomous mobile robotics. While numerous SLAM solutions exist, seamless integration between real-time mapping and dynamic path planning remains a critical engineering challenge. This paper presents a tightly coupled system that performs real-time 2D local SLAM and dynamic path planning simultaneously. The proposed architecture consists of two core components: (1) A 2D local SLAM module that incrementally builds a probability occupancy grid using log-odds updates and refines robot pose via Ceres-based scan matching; and (2) An A* path planner integrated with a modified Pure Pursuit controller that utilizes this continuously updated map to compute and execute optimal trajectories. The system was validated on a Clearpath Jackal platform. Experiments involving teleoperation-based mapping followed by autonomous navigation confirm that the robot successfully generates consistent local maps and dynamically adapts its trajectory to avoid obstacles. These results demonstrate that the proposed local SLAM system possesses sufficient fidelity and responsiveness to support autonomous navigation in dynamic environments.**

***Index Terms*—SLAM, Dynamic Path Planning, Ceres Scan Matching, A* Algorithm, Pure Pursuit Controller, Mobile Robotics, Probabilistic Robotics, ROS, Docker Containerization**

I. Introduction

Operating in unknown environments is a core challenge in mobile robotics. To achieve autonomy, a robot must estimate its location while simultaneously building a map of its surroundings. This interdependence is known as Simultaneous Localization and Mapping (SLAM).

While robust open-source packages like Gmapping [1] and Google Cartographer [2] exist, they focus solely on state estimation and mapping while operating independently from the path planning layer. Consequently, standard navigation frameworks such as ROS move_base [3] typically rely on pre-built, static maps. The primary objective of this work is to bridge this gap by implementing a unified system that executes real-time 2D SLAM and dynamic path planning simultaneously.

This paper presents a custom software stack validated on a Clearpath Jackal robot. We implemented a non-linear optimization backend using Google's Ceres Solver to fuse data from wheel odometry, an IMU, and a 2D LiDAR. Unlike static navigation approaches, our system couples this SLAM module directly with an A* path planner and Pure Pursuit controller. This integration allows the robot to dynamically adapt its trajectory as it builds the map in real-time.

II. Related Work

SLAM has been extensively studied in probabilistic robotics. Early approaches utilized the Extended Kalman Filter (EKF) to linearize pose prediction via first-order Taylor expansion [4]. However, EKFs often struggle with fast-changing non-linear dynamics which can lead to divergence and map drift. Subsequent methods employed Particle Filters (PF) for localization, most notably the FastSLAM algorithm [5]. Recently, increased computational capabilities have shifted the paradigm toward optimization-based approaches, such as Google Cartographer [2]. A comparative analysis by Tee and Han [6] highlights that these graph-based optimization methods generally offer superior accuracy and consistency in complex indoor environments compared to their filter-based counterparts.

Path planning within SLAM-generated maps is similarly well-established. Classical global planners like A* and Dijkstra remain prevalent due to their robust performance on occupancy grids [7]. For local motion control, geometric tracking algorithms such as Pure Pursuit have proven effective for differential-drive robots even on noisy paths [8]. More recent developments utilize Reinforcement Learning (RL); methods like QR-DQN have demonstrated efficacy in navigating complex, dynamic environments with imperfect sensor data [9].

III. System Overview

The Clearpath Jackal robot was used to facilitate the development of this project. The Jackal is a mobile ground robot platform designed for rapid prototyping and research applications [10]. We extended the base Jackal with an integrated Hoyoku UST-10 LX LiDAR mounted on the front of the robot.

The Jackal is built around a Nvidia Jetson running Ubuntu 20.04 paired with a 32-bit microcontroller (MCU). The MCU handles the IO, power supply monitoring, motor control, and data transmission from the integrated IMU and wheel encoders to the main computer via a full speed USB interface. The Hoyoku LiDAR connects directly to the Jetson computer using an ethernet interface.

The Jackal's software stack is built upon the Robot Operating System (ROS) Noetic distribution [11]. ROS is a modular framework consisting of *nodes*, which execute distinct functional tasks, and *topics*, which facilitate data transmission between them. Upon system initialization, the Jackal launches

Fig. 1: Clearpath Jackal mobile robot with mounted front LiDAR.

a suite of driver packages responsible for essential tasks such as odometry estimation and differential drive motion control.

We developed two ROS packages that integrate directly with the Jackal's existing software architecture. The SLAM package executes the *SLAM node* which localizes the robot within a dynamically updated map and publishes this spatial data. The planning package launches the *planning node* which subscribes to these map updates to generate optimal trajectories and publishes velocity commands to the Jackal's motor controller.

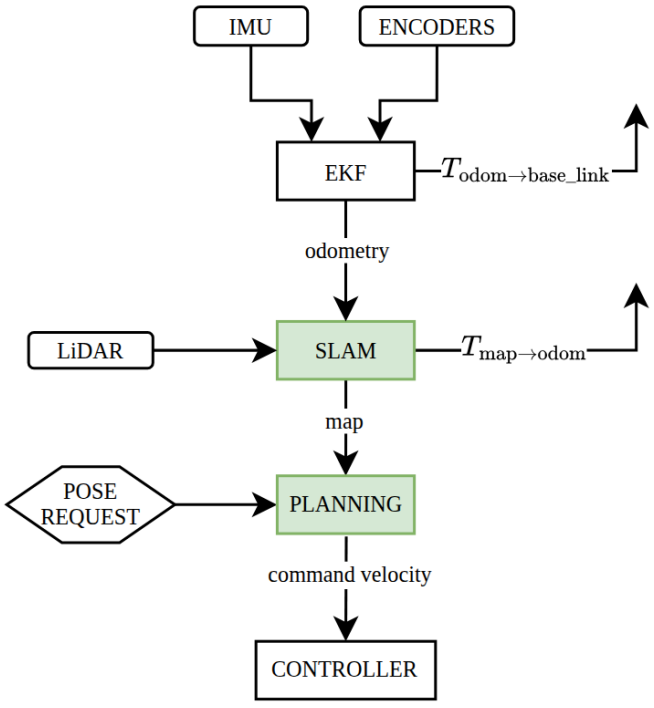


Fig. 2: Software architecture overview of the local 2D SLAM and dynamic path planning system. Green modules indicate the custom ROS packages developed for this project.

To ensure reproducibility and consistency across development machines, the entire software stack was containerized using Docker. Given the limited computational resources of the Jackal's onboard Nvidia Jetson, we implemented a distributed ROS architecture over a high-bandwidth Local Area Network (LAN) connecting the robot to a remote workstation (laptop).

In this configuration, the onboard Jetson functions as the ROS Master. The Jetson runs the low-level Jackal base drivers and publishes raw sensor and odometry data on the network. Computationally intensive tasks, specifically the *SLAM* and *planning nodes*, are offloaded to the workstation configured as a ROS Client. This off-board processing strategy optimizes resource allocation and enables higher-fidelity SLAM performance.

For preliminary validation, we utilized the Gazebo physics simulator [12]. The simulation environment was configured to mirror the physical robot [13], including the specific LiDAR specifications. This enabled rapid algorithmic testing in a virtual environment prior to real-world deployment.

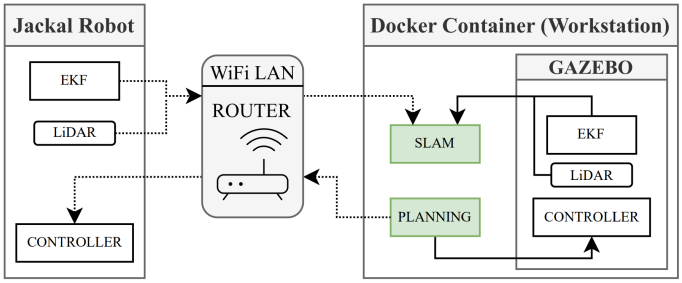


Fig. 3: Distributed system deployment diagram. The architecture leverages a Docker container on a remote workstation to handle computationally intensive SLAM and path planning tasks. The workstation communicates with the Jackal robot via a high-bandwidth local area network. For rapid testing, the Gazebo simulator is utilized on the workstation computer to emulate the behavior of the physical robot.

IV. Local 2D SLAM

Our SLAM system implements a high-resolution, real-time 2D LiDAR approach based on the methods described in [2]. Diverging from particle filter approaches that maintain a pose distribution, our system continuously optimizes a single robot pose in the map frame by matching incoming LiDAR scans against the current map. We formulate this scan matching process as a non-linear least squares error optimization problem.

The system is architected according to ROS REP-105 [14] standards to decouple local control from global navigation. An onboard EKF fuses inertial and encoder data to maintain the high-frequency, continuous local odometry $T_{\text{odom}\rightarrow\text{base\_link}}$ transform. Concurrently, the *SLAM node* computes the correction $T_{\text{map}\rightarrow\text{odom}}$ transform to mitigate accumulated drift introduced by the EKF. The SLAM system loops through three main stages to achieve this task: **pose prediction**, **scan optimization**, and **map integration**.

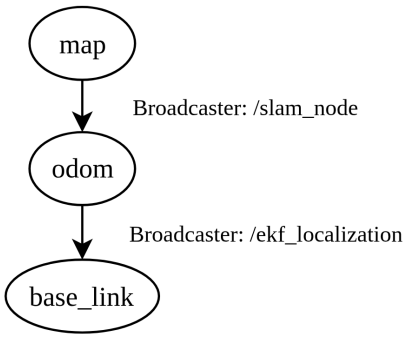


Fig. 4: ROS system transform tree. The odom frame is a continuous, short-term local reference but drifts over time. The map frame is a useful long-term global reference but discrete jumps make it a poor frame for local sense and act.

A. Probability Grid Representation

The environment is discretized into a 2D occupancy grid map, M. Each cell in the grid represents a specific area in the physical world and stores a probability value $p \in [0, 1]$ indicating the likelihood that the space is obstructed. The grid is initialized with a spatial resolution of 2 cm per cell.

Prior to observation, all cells are initialized to a sentinel value (-1) to represent the unknown state. As the robot traverses the environment, these values are updated to valid probabilities by integrating incoming LiDAR scans.

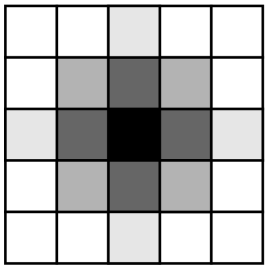


Fig. 5: Visualization of the occupancy grid representation of a circular object. The map encodes occupancy likelihood on a range of $[0, 1]$. Darker cells indicate a high probability of obstruction ($p \to 1$) while lighter cells represent free space ($p \to 0$).

B. Pose Prediction

The non-linear optimization problem is sensitive to local minima and requires a robust initial estimate to converge correctly. We leverage the robot's EKF odometry to generate this prediction.

At each step k, the relative transformation $\Delta T_{\text{base\_link}}$ is calculated between the current odometry transform and the previous odometry transform:

$$\Delta T_{\text{base\_link}} = T^{-1}_{\text{odom}\to\text{base\_link},k-1} \cdot T_{\text{odom}\to\text{base\_link},k} \quad (1)$$

Next, we apply this differential motion to the robot's previously optimized pose in the map frame. This yields the predicted initial estimate, $\hat{T}_{\text{map}\to\text{base\_link}}$, for the current time step k:

$$\hat{T}_{\text{map}\to\text{base\_link}} = T_{\text{map}\to\text{base\_link},k-1} \cdot \Delta T_{\text{base\_link}} \quad (2)$$

$\hat{T}_{\text{map}\to\text{base\_link}}$ provides the solver with a starting point close to the true solution.

C. Scan Optimization

Our objective is to find the optimal rigid body transformation $T^*_{\text{map}\to\text{base\_link}}$ that aligns scan points in the base_link frame with the high-probability occupancy grid cells in the map M. We formulate this as a non-linear least squares optimization problem [2], solved using Google's Ceres Solver [15].

The optimization minimizes the sum of squared residuals:

$$T^*_{\text{map}\to\text{base\_link}} = \underset{T}{\text{argmin}} \sum_{i=1}^{N} (1 - M_{smooth}(T \cdot h_i))^2 \quad (3)$$

where N is the number of points in the scan and h_i is the i-th scan point in the robot's base_link frame. T transforms the scan point h_i into the map frame.

Gradient-based solvers require the cost function to be continuous and differentiable. However, our occupancy grid M is discrete. We implement bicubic spline interpolation, denoted M_{smooth}, over the probability grid [2] and initialize the optimization with the predicted estimate $\hat{T}_{\text{map}\to\text{base\_link}}$ derived in Section IV-B.

To increase robustness against dynamic obstacles and sensor noise that introduce outliers, we apply a Cauchy loss function to the residuals. This kernel reduces the influence of large residuals and prevents transient obstacles from corrupting the state estimation or distorting the map.

D. Map Integration

Upon convergence of the scan optimizer, the scan points are transformed from the base_link frame to the map frame using the optimal transform $T^*_{\text{map}\to\text{base\_link}}$. These global points are then integrated into the occupancy grid M.

We identify two disjoint sets of grid cells for the update process [2]. The set of *hits* comprises the grid cells containing the transformed scan endpoints. The set of *misses* represents free space and is computed by ray-casting from the robot position in the map to each hit point using Bresenham's line algorithm.

The probability p of a cell is updated via the recursive binary Bayes filter.

$$\text{odds}(p) = \frac{p}{1-p} \quad (4)$$

$$p_{new} = \text{odds}^{-1}(\text{odds}(p_{old}) \cdot \text{odds}(p_{obs})) \quad (5)$$

where p_{obs} is the observation probability ($p_{hit} = 0.95$ for *hits*, $p_{miss} = 0.45$ for *misses*). Following [2], we employ a clamping function to ensure probabilities remain within the bounds $[0.001, 0.999]$. This prevents the system from becoming overconfident and failing to update the map when the environment changes.

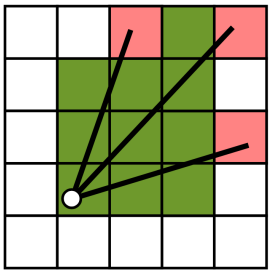


Fig. 6: Visualization of the map update logic. The set of *hits* (red) denotes object detections, while the set of *misses* (green) represent free space derived via ray-casting from the robot position (circle).

E. Drift Correction

Using the optimized transform $T^{*}_{\text{map}\to\text{base\_link}}$, the system closes the loop by computing the correction transform between the map and odometry frames, denoted as $T_{\text{map}\to\text{odom}}$. This step realigns the global reference frame to counter accumulated drift without introducing discontinuities into the local odometry stream.

$$T_{\text{map}\to\text{odom}} = T^{*}_{\text{map}\to\text{base\_link}} \cdot T^{-1}_{\text{odom}\to\text{base\_link},k} \tag{6}$$

The resulting $T_{\text{map}\to\text{odom}}$ is broadcast to the ROS transform tree providing a corrected global reference for high-level navigation and path planning.

V. Path Planning and Control

The navigation system is designed to generate and execute collision-free trajectories within the environment mapped by our SLAM module. The *planning node* adopts a hierarchical control architecture consisting of two distinct layers to achieve autonomous navigation from the robot's estimated pose to a user-defined target in the map.

First, a global planner operates over the occupancy grid map to compute an optimal, obstacle-free path using the A* search algorithm. Second, a local motion controller receives this path and generates velocity commands using the Pure Pursuit algorithm to ensure precise trajectory tracking. This architecture is realized by multi-threading the global planner and the motion controller within the *planning node*.

A. Global Path Planning using A*

By subscribing to the occupancy grid map M published by the *SLAM node*, we discretize the environment into traversable and non-traversable cells. Unknown cells (value -1) are treated as free space to facilitate exploration. To account for the robot's physical dimensions, we transform the map into configuration space by inflating obstacles by the robot's radius.

Each cell is treated as a node in a uniform 8-connected graph, where edges only exist between free cells. Given the robot's current estimated pose (x_e, y_e) and requested pose (x_r, y_r), we run A* search to compute a minimum-cost path. For each node n, the evaluation function is defined as

$$f(n) = g(n) + h(n) \tag{7}$$

where $g(n)$ denotes the accumulated cost from the start node to n, and $h(n)$ is the heuristic estimate of the remaining cost from n to the requested pose node [16]. In this work, since the robot is running in a 2D plane, it is intuitive to use a Euclidean distance between n and the goal as the heuristic funtion. During planning, nodes are expanded in ascending order of $f(n)$ using a priority queue, while already visited nodes are stored in a closed list. The search terminates when the goal node or a neighborhood cell within one stride is reached, after which the final path is reconstructed by backtracking through parent pointers. We then smooth the waypoints by a gradient-based method balancing the fidelity of the original path with a smoothness penalty on the second derivative. The smoothed waypoint sequence is finally returned as the trajectory for the controller to follow.

The high-level procedure of A* planner used in this work is summarized in Algorithm 1.

Algorithm 1 Global Path Planning using A* Search

Input: Start pose p_s, goal pose p_g, occupancy grid M
Output: Waypoint sequence $\mathcal{P}$

```
Inflate obstacles in map M
Convert p_s, p_g into grid indices
Initialize open list (priority queue) and closed list
Insert start node with g = 0 and f = h
while open list not empty do
    n ← pop node with smallest f
    if n is at or within stride of p_g then
        Backtrack through parent pointers
        Smooth path P
        return path P
    end if
    Mark n as closed
    for each 8-connected neighbor v of n do
        if v is free and not closed then
            Update g(v) and f(v) if improvement found
            Push v back into open list
        end if
    end for
end while
return Failure
```

B. Pure Pursuit Controller

To execute the trajectory generated by the A* global planner, we implement a path-tracking controller derived from the standard Pure Pursuit algorithm [17]. While the classic formulation calculates the curvature required to move the robot from its current pose to a discrete waypoint on the path, we introduce a continuous goal point selection to mitigate oscillations caused by path discretization [18].

Given the current robot pose in the map and a precomputed path, our controller will determine the necessary angular velocity for the robot to follow the path. Linear velocity is treated as constant so a simple proportional(p)-controller was

added to slow the robot down as it approaches the endpoint.

1) Lookahead Goal Selection: The controller identifies a target goal point (x_g, y_g) on the path at a fixed lookahead distance l_d away from the robot. The standard implementation proposed by [17] selects the nearest discrete waypoint on the path at distance l_d which can result in unstable steering control [18]. To address this, we implement the *Linear Interpolation* method proposed by Ohta et al. [18].

We calculate the exact geometric intersection between a search circle of radius l_d centered at the robot and the linear segments connecting consecutive path waypoints. To ensure computational efficiency, we employ a sliding window search strategy that initializes the intersection check at the index of the previously identified path segment.

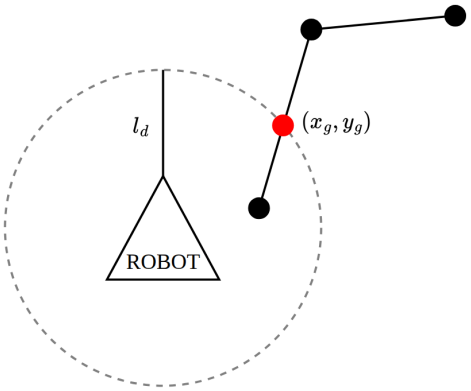


Fig. 7: Visualization of Pure Pursuit controller goal point selection. The calculated goal point (x_g, y_g) (shown in red) on the path is a certain lookahead l_d distance away from the robot.

The lookahead distance l_d acts as a gain on the tracking response. As described by Coulter, the system dynamics resemble a second-order system where l_d influences the damping [17]. A small l_d provides tight path adherence but increases the risk of oscillation. Conversely, a large l_d results in smoother motion but causes the robot to drive significantly inside the curve (under-steer) at sharp turns [17], [18].

2) Velocity Control Law: Once the goal point is established, velocity commands are generated using a p-control scheme.

We deviate from the standard geometric curvature calculation defined in [17]. Instead, we calculate the angular velocity ω_{cmd} by minimizing the heading error between the robot's current yaw θ_{curr} and the bearing to the goal point θ_{target}:

$$\omega_{cmd} = K_\omega \cdot (\theta_{target} - \theta_{curr}) \tag{8}$$

where K_ω is a tunable proportional gain.

To ensure safe deceleration, the linear velocity v_{cmd} is scaled proportionally to the Euclidean distance between the robot position P_{robot} and the current goal point P_{goal}:

$$v_{cmd} = K_v \cdot ||P_{goal} - P_{robot}|| \tag{9}$$

where K_v is a tunable proportional gain.

During standard path following, the distance between the robot and the goal point remains constant at approximately l_d. This results in a constant tracking speed. However, as the robot approaches the final waypoint on the path, the robot decelerates to a stop.

Both v_{cmd} and ω_{cmd} are limited to a maximum linear and angular speed determined by configurable Pure Pursuit controller parameters.

VI. Experiments

In this section, we want to evaluate the performance of our local SLAM and path planning algorithms. Experiments in both the Gazebo simulator [13] and the real world are conducted to validate sim-to-real transferability. Qualitative results are visualized using RViz (ROS Visualization).

A. Simulation

SLAM and path planning experiments were conducted within a controlled Gazebo simulation environment. The simulator environment was configured to replicate the physical characteristics of the real world, serving as a baseline for validating algorithmic performance prior to physical deployment. The simulated environment is depicted in Figure 8.

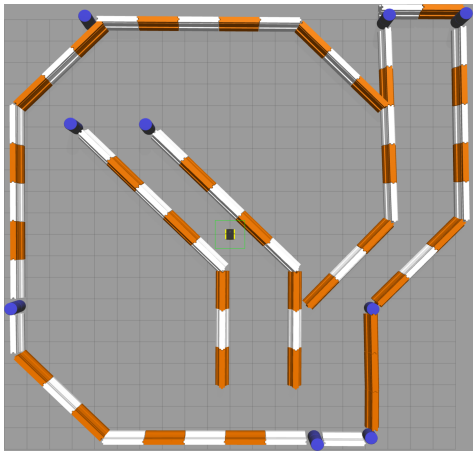


Fig. 8: Gazebo simulation testing environment.

1) SLAM Tests: To evaluate the SLAM algorithmic performance, we drove the simulated Jackal robot around the environment and measured the generated occupancy grid (shown in Figure 9. To quantitatively evaluate our mapping results, we compared the SLAM-generated occupancy map with the ground truth map from the Gazebo environment. After binary extraction of the structural obstacles, the SLAM map achieved a recall of 0.73. The high recall indicates that around 73% of the structural obstacles present in the environment were successfully reconstructed in our generated SLAM map. As illustrated in Figure 9, the occupied cells (black) align closely with the ground truth obstacle boundaries (green).

To quantify the geometric fidelity of the reconstructed map, we computed the Euclidean distance between ground-truth boundary pixels and the nearest SLAM-generated obstacles.

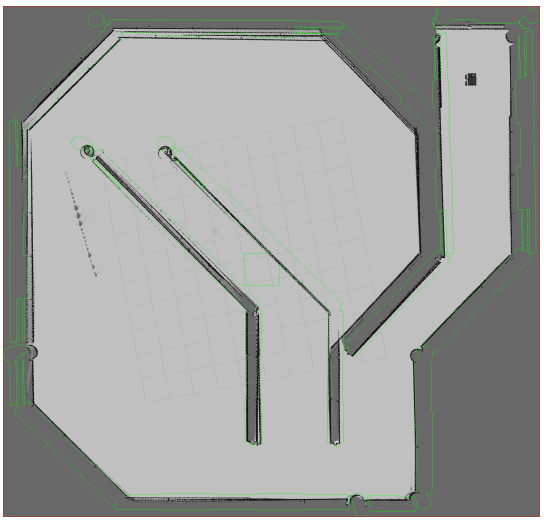


Fig. 9: Occupancy grid generated by the SLAM algorithm. Green lines represent the ground truth obstacle boundaries from the simulation environment.

The error distribution exhibits a mean deviation of 3.09 pixels and a median of 0. While the maximum outlier deviation reached 68.59 pixels, the 95th-percentile error remained limited to 20.47 pixels. The combination of a zero median deviation and a structural recall of 0.73 indicates that the majority of the environment's static features were reconstructed with high spatial accuracy, with errors primarily confined to outliers.

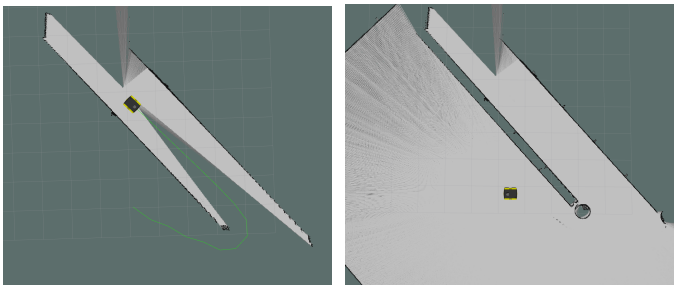
TABLE I: Statistical distribution of geometric deviation between SLAM-reconstructed walls and ground truth.

Metric	Deviation (pixels)
Mean	3.094
Median	0.000
90th Percentile	11.909
95th Percentile	20.469
Maximum	68.591

2) Planning Tests: Path planning validation was conducted within the same simulation environment. We performed three independent trials targeting a goal pose displaced 4 meters from the robot's initial position. Notably, the robot exhibited frontier exploration behavior, successfully avoiding an unmapped obstacle to reach the target (Figure 10). The terminal Euclidean errors for the trials were recorded as 0.3 m, 0.7 m, and 0.4 m, respectively. The resulting mean error of 0.47 m aligns closely with the controller's configured goal tolerance of 0.4 m. These results demonstrate the repeatability and precision of the navigation stack in reaching requested poses.

B. Live Demonstration

A live demonstration of the proposed system was conducted on the first floor of the Computing Science Centre (CSC) at the University of Alberta. The experiments aimed to qualitatively assess the fidelity of the SLAM module and the responsiveness of the integrated path planning architecture.



(a) Target goal specified. (b) Target goal reached.

Fig. 10: Simulation validation of the path planning subsystem. (a) visualizes the global path (green) dynamically adjusting to avoid obstacles. (b) depicts the robot successfully arriving at the target pose within the specified tolerance.

The first phase of testing isolated the SLAM system to evaluate its ability to construct a consistent global map. The Jackal robot was tele-operated through the building while the *SLAM node* performed real-time scan matching and occupancy grid updates.

Figure 11 presents the resulting occupancy grid. The map is constructed with high structural integrity; long corridors are reconstructed as straight features and corners are distinct rather than rounded. This visual evidence suggests the Ceres-based scan matching successfully minimized angular drift which is a common failure mode in deadreckoning odometry systems.

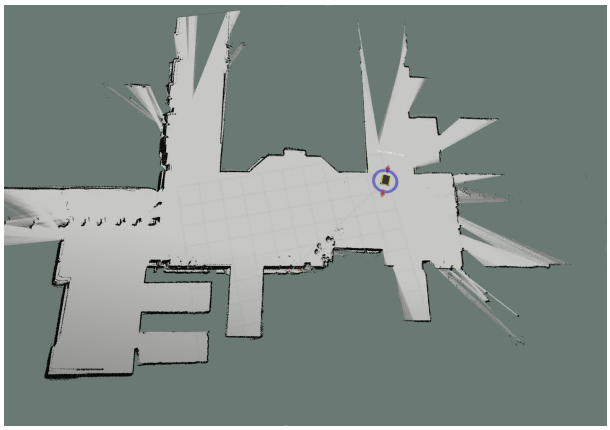
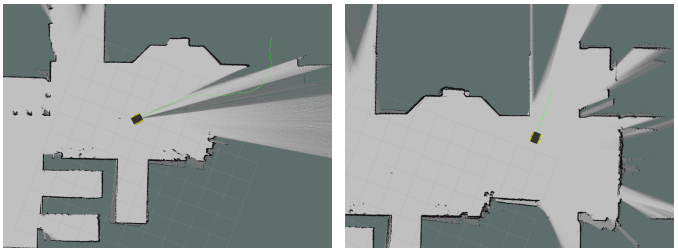


Fig. 11: Occupancy grid map of the CSC first floor generated by the custom SLAM implementation.

The second phase of testing validated the integration of the SLAM module with the A* planner and Pure Pursuit controller. In this configuration, the robot was tasked with autonomously navigating to user-defined goal poses.

Figure 12a demonstrates the system's capability to plan trajectories into previously unobserved areas. The green visualization line represents the optimal path computed by the A* global planner. Despite the lack of map data at the requested pose, the planner successfully treated unknown space as traversable which allowed the robot to expand the map boundaries and explore the unknown frontier.

Figure 12b is a continuation of Figure 12a at a later time step. It illustrates navigation within a fully mapped environment. Here, the global planner generated a collision-free trajectory through the corridor. The path maintains a safety margin from the walls. This margin verifies that the obstacle inflation parameters are working and tuned correctly. Throughout the traversal, the robot dynamically updated its path in response to the real-time map updates provided by the SLAM system and successfully reached the target pose without collision.



(a) Planning into unknown space. (b) Navigation in mapped area.

Fig. 12: Demonstration of planning operation. (a) shows frontier exploration where the A* planner generates a trajectory (green line) into unknown space which allows the robot to autonomously explore and expand the map. (b) shows goal pose navigation within a fully mapped area where the green path indicates the optimal trajectory to the goal pose.

VII. Conclusion

This paper presented an integrated local 2D SLAM and dynamic path planning system validated on the Clearpath Jackal robot platform. By fusing Ceres-based scan matching with hierarchical planning, we enabled robust autonomous exploration without a pre-existing map. Experimental results confirmed high mapping fidelity, achieving a structural recall of 0.73, while dynamic path planning successfully navigated the unknown frontier. Future work will address long-term drift by implementing graph-based loop closure in the SLAM algorithm.

Acknowledgment

The authors would like to thank the University of Alberta Computer Vision and Robotics Laboratory for providing access to the Jackal robot. Special thanks to Professor Martin Jagersand for his guidance throughout the CMPUT 312 course, and to the Teaching Assistants, Justin Valentine and Riley Zilka, for their valuable support and feedback.

References

[1] "OpenSLAM.org," openslam-org.github.io. https://openslam-org.github.io/gmapping.html

[2] W. Hess, D. Kohler, H. Rapp and D. Andor, "Real-time loop closure in 2D LIDAR SLAM," 2016 IEEE International Conference on Robotics and Automation (ICRA), Stockholm, Sweden, 2016, pp. 1271-1278, doi: 10.1109/ICRA.2016.7487258.

[3] "move_base - ROS Wiki," wiki.ros.org. https://wiki.ros.org/move_base

[4] M. I. Ribeiro, "Kalman and extended Kalman filters: Concept, derivation and properties," Institute for Systems and Robotics, vol. 43, no. 46, pp. 3736–3741, 2004.

[5] M. Montemerlo and S. Thrun, "FastSLAM: A Scalable Method for the Simultaneous Localization and Mapping Problem in Robotics (Springer Tracts in Advanced Robotics) ", Springer-Verlag, 2007.

[6] Y. K. Tee and Y. C. Han, "Lidar-Based 2D SLAM for Mobile Robot in an Indoor Environment: A Review," 2021 International Conference on Green Energy, Computing and Sustainable Technology (GECOST), Miri, Malaysia, 2021, pp. 1-7, doi: 10.1109/GECOST52368.2021.9538731.

[7] P. E. Hart, N. J. Nilsson and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," in IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100-107, July 1968, doi: 10.1109/TSSC.1968.300136.

[8] M. Samuel, M. Hussein, and M. Binti, "A review of some pure-pursuit based path tracking techniques for control of autonomous vehicle," Int. J. Comput. Appl., vol. 135, no. 1, pp. 35–38, Feb. 2016.

[9] M. G. Bellemare, S. Candido, P. S. Castro, J. Gong, M. C. Machado, S. Moitra, S. S. Ponda, and Z. Wang, "Autonomous navigation of stratospheric balloons using reinforcement learning," Nature, vol. 588, no. 7836, pp. 77–82, Dec. 2020, doi: 10.1038/s41586-020-2939-8.

[10] "Jackal User Manual — Clearpath Robotics Documentation," Clearpathrobotics.com, Mar. 25, 2025. https://docs.clearpathrobotics.com/docs_robots/legacy/ros1_robots/outdoor_robots/jackal/user_manual_jackal (accessed Dec. 07, 2025).

[11] "Documentation - ROS Wiki," Ros.org, 2018. https://wiki.ros.org/

[12] Open Robotics, "Gazebo," gazebosim.org. https://gazebosim.org/home

[13] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," 2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566), Sendai, Japan, 2004, pp. 2149-2154 vol.3, doi: 10.1109/IROS.2004.1389727.

[14] W. Meeussen, "REP 105 – Coordinate Frames for Mobile Platforms (ROS.org)," Ros.org, 2025. https://www.ros.org/reps/rep-0105.html

[15] Agarwal, K. Mierle, and The Ceres Solver Team, "Ceres Solver," version 2.2, Oct. 2023. [Online]. Available: https://github.com/ceres-solver/ceres-solver

[16] A. Patel, "Introduction to A*," theory.stanford.edu, 1997. https://theory.stanford.edu/~amitp/GameProgramming/AStarComparison.html (accessed Dec. 07, 2025).

[17] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Robotics Institute, Pittsburgh, PA, Tech. Rep. CMU-RI-TR-92-01, January 1992.

[18] H. Ohta, N. Akai, E. Takeuchi, S. Kato and M. Edahiro, "Pure Pursuit Revisited: Field Testing of Autonomous Vehicles in Urban Areas," 2016 IEEE 4th International Conference on Cyber-Physical Systems, Networks, and Applications (CPSNA), Nagoya, Japan, 2016, pp. 7-12, doi: 10.1109/CPSNA.2016.10.